In [15]:
```
import pandas as pd

df = pd.read_csv('./Desktop/TPs ML/TP02/data.csv')
```

In [17]:
```
df.head()
```

Out[17]:

	pclass	survived	name	sex	age	sibsp	parch	ticket	fare	cabin
0	1.0	1.0	Allen, Miss. Elisabeth Walton	female	29.0000	0.0	0.0	24160	211.3375	B5
1	1.0	1.0	Allison, Master. Hudson Trevor	male	0.9167	1.0	2.0	113781	151.5500	C22 C26
2	1.0	0.0	Allison, Miss. Helen Loraine	female	2.0000	1.0	2.0	113781	151.5500	C22 C26
3	1.0	0.0	Allison, Mr. Hudson Joshua Creighton	male	30.0000	1.0	2.0	113781	151.5500	C22 C26
4	1.0	0.0	Allison, Mrs. Hudson J C (Bessie Waldo Daniels)	female	25.0000	1.0	2.0	113781	151.5500	C22 C26

In [23]:
```
df.shape
```

Out[23]:
```
(1310, 14)
```

In [25]:
```
df.columns
```

Out[25]:
```
Index(['pclass', 'survived', 'name', 'sex', 'age', 'sibsp', 'parch', 'ticke
t',
       'fare', 'cabin', 'embarked', 'boat', 'body', 'home.dest'],
      dtype='object')
```

In [27]:
```
len(df.columns)
```

Out[27]:
```
14
```

In [39]:
```
print('nomber des individu c\'est',df.shape[0])
```

```
nomber des individu c'est 1310
```

In [41]:
```
print('les caractéristiques c\'est',df.shape[1])
```

```
nomber des variable c'est 14
```

we are looking the dataset of titancs so the targeted variable should be if the individual survived or not which in this case **survived**

as we can see for the first 5 individual there are missing values

1.3-Compter le nombre de valeurs dans la colonne age possédant des valeurs manquantes :

1. Assigner à la variable age la colonne des âges du dataframe titanic_survival
2. Utiliser pandas.isnull() sur la variable age pour créer une Series de valeurs True et False
3. Utiliser la Series résultante pour sélectionner seulement les éléments de la colonne age qui sont nuls et assigner le résultat à la variable missing_values
4. Assigner le nombre de valeurs manquantes de missing_values à la variable missing_values_count (fonction len())
5. Afficher missing_values_count pour voir le nombre de valeurs manquantes de la colonne age.

In [263…
```
# 1
age=df['age']
age.head()
```

Out[263…
```
0    29.0000
1     0.9167
2     2.0000
3    30.0000
4    25.0000
Name: age, dtype: float64
```

In [68]:
```
# 2
# pandas=pd
age.isnull()
```

Out[68]:
```
0       False
1       False
2       False
3       False
4       False
        ...
1305     True
1306    False
1307    False
1308    False
1309     True
Name: age, Length: 1310, dtype: bool
```

as we can see the predefined function on pandas `isnull()` we give us the table it return **False** if there is a value and it return **TRUE** if there isn't

In [74]:
```
# 3
missing_values= age[age.isnull()]
missing_values.head()
```

Out[74]:
```
15   NaN
37   NaN
40   NaN
46   NaN
59   NaN
Name: age, dtype: float64
```

we have selected the missing values from our dataset and we see that the first five missing values in our dataset are the indivdual 15,37,40,46, and 59

In [91]:
```
# 4
nbr_missing_values=len(missing_values)
```

we used the function `len()` to calculte the length of our new table of missing values.

In [84]:
```
# 5

print(" le nombre de valeurs manquantes de la colonne age",nbr_missing_values)
```

```
le nombre de valeurs manquantes de la colonne age 264
```

1.4-Faire de même avec la colonne 'cabin' avec l'instruction isnull().sum()

In [93]:
```
cabin =df['cabin']
```

We created a new Series called cabin that contains the values of the cabin column.

In [96]:
```
cabin.isnull().sum()
```

Out[96]:
```
1015
```

this way all the work we did previously is simplified in simple way create a new serie with `isnull()` and then summing the **TRUE** values with the function `sum()` .

1.5-Compter les valeurs manquantes pour chaque colonne.

to do that we will apply what we did to the column cabin to all the dataframe

In [108…
```
all_missing_values=df.isnull().sum()

print(all_missing_values)
```

```
pclass          1
survived        1
name            1
sex             1
age           264
sibsp           1
parch           1
ticket          1
fare            2
cabin        1015
embarked        3
boat          824
body         1189
home.dest     565
dtype: int64
```

and if we want to know how many missing values are there we just have to sum the previous one

In [113…
```
all_missing_values.sum()
```

Out[113…
```
3869
```

as we can see there are 3869 missing values in our dataframe

1.6-Discuter l'importance de gérer les données manquantes et les différents moyens de le faire.

it's neccessary to manage the missing values in our dataframe because it will effect our etude in we decide to explore this dataframe or deploy machine learning model.

2- Gérer les variables manquantes

2.1 Supprimer les lignes contenant des valeurs manquantes dans la colonne 'embarked'.

In [126…
```
df = df[df['embarked'].notna()]
```

we can check if there is no 'NaN' values in our dataframe

In [145…
```
df['embarked'].isnull().sum()
```

Out[145…
```
0
```

In [153…
```
??notna()
```

```
Object `notna()` not found.
```

2.2 Supprimer la colonne 'cabin' du dataset.

we will simply use the function `drop()`

In [ ]:
```
df = df.drop(columns=['cabin'])
```

2.3 Imputer les valeurs manquantes:

1. Imputation des variables numériques : Remplacer les valeurs manquantes de la colonne 'age' par la moyenne des âges.
2. Imputation des variables catégoriques : Remplacer les valeurs manquantes de la colonne 'embarked' par la valeur la plus fréquente (mode). (Utilisez la stratégie 'most_frequent')

In [285…
```
# 1
mean_age=age.sum()/len(age[age.notna()])
```

In [ ]:
```
len(age[age.notna()])
# les valeurs no null
```

we calculted the sum of the Series and then we divide it by the length on existing values we exculed the NaN values with this expression `age[age.notna()]`

In [292…
```
mean_age
```

Out[292…
```
29.850348184214724
```

In [294…
```
age[age.isnull()]=mean_age
```

```
C:\Users\AL-Athir\AppData\Local\Temp\ipykernel_13516\172436598.py:1: SettingWit
hCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/sta
ble/user_guide/indexing.html#returning-a-view-versus-a-copy
  age[age.isnull()]=mean_age
```

we affected the mean in every NaN values

In [307…
```
# 2
embarked = df['embarked']
```

we will use the predfined function `unique()` to see the uniques values in the series embarked

In [310…
```
embarked.unique()
```

Out[310…
```
array(['S', 'C', 'Q'], dtype=object)
```

as we can see there are three possible values in the column embarked let's see their frequency

In [346…
```
len([item for item in embarked if item == 'S'])
```

Out[346…
```
914
```

In [348…
```
len([item for item in embarked if item == 'C'])
```

Out[348…
```
270
```

In [350…
```
len([item for item in embarked if item == 'Q'])
```

Out[350… 123

as we can see that the value that is more frequent is **S** with a repetition of 914

In [364…
```
df[df['embarked'].isnull()]= 'S'
```

3- Gérer les variables catégoriques

1. En considérant la variable 'embarked' comme indépendante, appliquez l'encodage nécessaire (par exemple, LabelEncoder ou OneHotEncoder) pour remplacer la colonne 'embarked' d'origine dans le DataFrame Titanic par les nouvelles colonnes encodées. Écrivez le code nécessaire pour effectuer cette opération et expliquez pourquoi l'encodage choisi est important pour cette variable.

In [384…
```
from sklearn.preprocessing import LabelEncoder


encoder = LabelEncoder()

df['embarked'] = encoder.fit_transform(df['embarked'])

df.head()
```

Out[384…

	pclass	survived	name	sex	age	sibsp	parch	ticket	fare	emba
0	1.0	1.0	Allen, Miss. Elisabeth Walton	female	29.0000	0.0	0.0	24160	211.3375	
1	1.0	1.0	Allison, Master. Hudson Trevor	male	0.9167	1.0	2.0	113781	151.5500	
2	1.0	0.0	Allison, Miss. Helen Loraine	female	2.0000	1.0	2.0	113781	151.5500	
3	1.0	0.0	Allison, Mr. Hudson Joshua Creighton	male	30.0000	1.0	2.0	113781	151.5500	
4	1.0	0.0	Allison, Mrs. Hudson J C (Bessie Waldo Daniels)	female	25.0000	1.0	2.0	113781	151.5500	

2. En considérant la variable sex comme dépendante, appliquez l'encodage nécessaire à cette colonne pour qu'elle puisse être utilisée dans un modèle de machine learning.

In [389…
```
df['sex'].unique()
```

Out[389…
```
array(['female', 'male'], dtype=object)
```

as we can see it's binary we can do 1 for male and 0 for female

In [391…
```
df['sex']=(df['sex']=='male').astype(int)
```

Conclusion

In this TP, we learned how to import and explore a DataFrame using the powerful Pandas library. We began by familiarizing ourselves with the dataset structure and identifying key features, as well as understanding the importance of handling missing data effectively. We saw that missing data can significantly impact the accuracy and performance of machine learning models, and it is crucial to address it properly.

To manage missing data, we applied different techniques depending on the nature of the data. For numerical variables, such as the 'age' column, we imputed missing values with the mean, ensuring that the data remained representative and usable for analysis. For categorical variables, like the 'embarked' column, we replaced missing values with the most frequent category (mode), ensuring that no valuable information was lost.

Additionally, we explored how to encode categorical variables to make them compatible with machine learning models. By using techniques such as one-hot encoding for independent categorical variables and binary encoding for dependent variables, we transformed textual data into numerical format that can be processed by machine learning algorithms.

Overall, this TP provided essential insights into the crucial preprocessing steps required before applying machine learning models. We learned how to import, explore, and clean data, manage missing values, and properly encode categorical variables. These steps are fundamental for building reliable models and ensuring that data is in an optimal format for analysis. In future projects, we can further enhance our data preprocessing by exploring other techniques like feature scaling and advanced imputation methods.